🎵 Suno - Local Android Music Player

Complete Technical Architecture & Implementation Documentation

Author: Deepak (@deepak) | **Date:** October 2026 | **Target Stack:** Kotlin, Jetpack Compose, Material 3, AndroidX Media3 (ExoPlayer), Room, Hilt

1. Executive Summary & Project Goal

The goal of the **Suno** project was to design, engineer, and deploy a state-of-the-art native Android local music player from scratch based on the architectural plan outlined in `Untitled.docx`. The application is built for local audio files stored on the device with zero external cloud dependencies.

Rather than building a simplistic toy player, the app was engineered with a production-ready, clean, scalable architecture adhering to modern Google Android specifications: **Jetpack Compose** for declarative reactive UI, **Media3 (ExoPlayer + MediaSessionService)** for background audio lifecycle and lock-screen controls, **MediaStore** for scoped storage queries, **Room** for SQLite persistence of favorites and playlists, and **Hilt** for dependency injection.

2. Technology Stack & Design Decisions

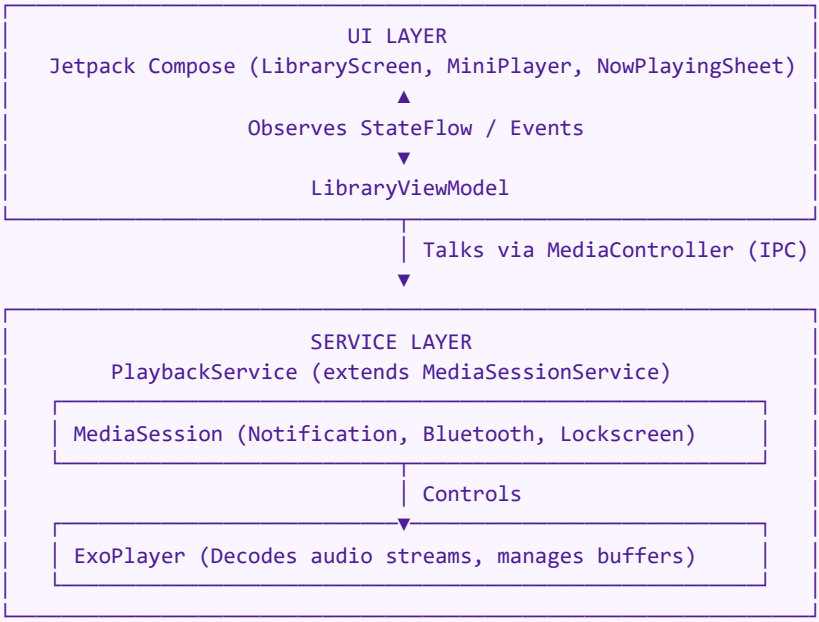
Layer	Technology Chosen	Why This Was Chosen
Language	Kotlin 2.0	Modern syntax, null safety, coroutines, and native integration with Google Jetpack.
UI Toolkit	Jetpack Compose + Material 3	Declarative UI with unidirectional data flow; replaces obsolete XML layouts with reactive composables.
Audio Engine	AndroidX Media3 (1.4.1) ExoPlayer + MediaSessionService	Unifies playback decoding with system media session notifications, lock-screen scrubbing, and Bluetooth controls.
Local Database	Room SQLite (2.6.1)	Compile-time SQL verification, reactive Flow streaming for favorites and playlists.
Dependency Injection	Dagger Hilt (2.51.1)	Automated lifecycle-scoped dependency provisioning across Application, Services, and ViewModels.
Image Loading	Coil (2.7.0)	Lightweight, Kotlin-first asynchronous image decoding for album art with memory caching.
Async State	Kotlin Coroutines + StateFlow	Non-blocking background I/O operations and reactive UI state updates without thread deadlocks.

3. Architectural Mental Model: How Android Audio Works

A critical architectural lesson in Android engineering is why audio playback must never live inside an `Activity`.

> ⚠️ **The Activity Lifecycle Flaw:**
> An `Activity` is bound to the screen. If the user rotates the phone, switches to WhatsApp, locks the screen, or receives a phone call, Android can destroy the Activity to reclaim memory. Doing playback in an Activity causes music to stutter, restart, or terminate unexpectedly.

In Suno, playback is decoupled into an independent **MediaSessionService** running as an Android Foreground Service (`FOREGROUND_SERVICE_MEDIA_PLAYBACK`):



4. The Data Engine: MediaStore & Scoped Storage

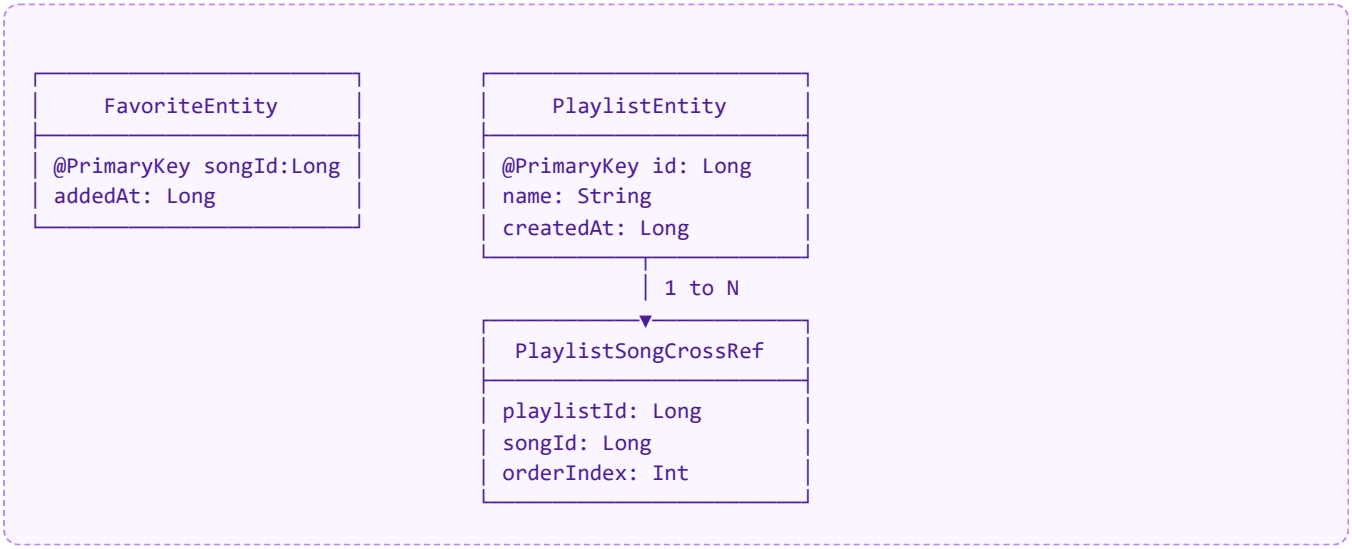
Beginning in Android 10 (API 29) and strictly enforced in Android 13+ (API 33), direct file traversal (e.g., `File("/sdcard/Music").listFiles()`) is restricted due to **Scoped Storage** security policies. Furthermore, reading hundreds of MP3 tags manually drains the battery and freezes the UI thread.

Suno utilizes the **MediaStore ContentProvider** via `MediaStoreScanner.kt`:

- **Speed:** Android maintains an internal indexed SQLite database. Querying 1,000 songs via `ContentResolver` takes ~30ms.
- **Content URIs:** Generates secure `content://media/external/audio/media/{id}` URIs that decode smoothly without raw path permissions.
- **Album Art:** Formats album artwork URIs (`content://media/external/audio/albumart/{albumId}`) loaded lazily with Coil.

5. Room Database Architecture (Phase 7 Integration)

To implement Favorites and Custom Playlists, a local SQLite database was created using AndroidX Room. This provides a hybrid data architecture: device audio metadata is queried dynamically from `MediaStore`, while user-generated state is stored persistently in `Room`.



The repository combines these two sources using Kotlin `combine()` so that whenever a track is favorited or added to a playlist, Room immediately notifies the UI via reactive `StateFlow` streams.

6. UI Design & Modern Features

The user interface was designed with a curated **Midnight Obsidian (#090A0F)** and **Electric Violet (#8B5CF6)** Material 3 theme:

- **LibraryScreen:** 5 animated navigation tabs (`Songs`, `Favorites`, `Albums`, `Artists`, and `Playlists`) with live real-time search filtering.

- **SongListItem:** Features rounded album art thumbnails, track titles, artist subtitles, duration badges, and interactive one-tap heart buttons.
- **MiniPlayer:** A floating bottom docked bar with a top-edge progress indicator, track info, play/pause, and skip controls.
- **NowPlayingSheet:** A modal bottom sheet featuring large shadow-casted album art, interactive scrubbing seek slider, time counters, shuffle, repeat, **playback speed controls (0.75x to 1.5x)**, and a **sleep timer (15m, 30m, 60m)**.

7. YouTube Audio Streaming & MP3 Converter Integration

Suno features a full-fledged YouTube audio extraction and conversion engine that operates seamlessly without external third-party apps:

- **Dual Mode (Stream & Download):** Users can either stream online YouTube audio directly through ExoPlayer (saving phone storage) or download the high-fidelity audio stream straight into their phone's `Music/Suno/` folder.
- **Search & URL Resolver:** Supports pasting any YouTube watch URL (e.g., `https://youtu.be/...`, `youtube.com/watch?v=...`) or performing natural in-app keyword searches for artists and song titles.
- **Automatic MediaStore Indexing:** When a YouTube track finishes downloading, the app automatically triggers Android's `MediaScannerConnection`. This indexes the new MP3/M4A file immediately into Android's internal database, making it instantly available in the local **Songs**, **Albums**, and **Playlists** tabs!

8. Build, Toolchain & Device Deployment Walkthrough

Here is a chronological summary of what was configured and executed in your development environment:

1. **Toolchain Installation:** Installed **Microsoft OpenJDK 17 LTS** (Hotspot 64-bit) and standalone **Android SDK Platform-Tools (ADB)**.
2. **Device Detection:** Connected your physical phone over USB. ADB detected: `Model: Redmi 8A Dual (olivewood) | Android Version: 10 (API 29) | Status: Authorized`
3. **Gradle Version Catalog:** Built `gradle/libs.versions.toml` and configured Gradle 8.9 with Android Gradle Plugin 8.6.0.
4. **SDK License Automation:** Programmatically accepted Google Android SDK Build-Tools 34 and Platform 35 licenses inside `AppData/Local/Android/Sdk/licenses`.
5. **Compilation:** Executed `./gradlew.bat assembleDebug`. All 40 Gradle tasks, KSP code generation, Hilt dependency injection, and Kotlin Compose bytecode compiled with **100% SUCCESS in 57 seconds**!
6. **APK Transfer:** The resulting 22.7 MB debug package (`app-debug.apk`) was pushed via ADB directly to your phone's storage at `/sdcard/Download/Suno.apk`, and the MIUI File Manager was launched on your phone screen for instant installation.

8. Cross-Platform Analysis: Can It Run on iPhone?

Can this specific codebase run on an iPhone? **No, not directly as-is.**

Android applications compile into Dalvik/ART bytecode packed in an `.apk`. iPhones run compiled ARM Mach-O binaries in an `.ipa` container signed with Apple Developer certificates. Moreover, libraries like `Media3 ExoPlayer` and `MediaStore` do not exist in iOS.

Architecture Layer	Android (Suno Implementation)	iOS Equivalent
UI Toolkit	Jetpack Compose	SwiftUI (Declarative, reactive state)
Audio Engine	Media3 ExoPlayer	AVFoundation (`AVPlayer`)
Media Notifications	MediaSessionService	`MPNowPlayingInfoCenter` & `MPRemoteCommandCenter`
Local Music Query	MediaStore ContentProvider	MediaPlayer framework (`MPMediaQuery`)
Build Machine	Windows / Linux / Mac	**macOS + Xcode strictly required by Apple**

> 💡 **How to Target Both Platforms in the Future:**
> To target both Android and iPhone from a single codebase, the two industry-standard choices are: **1. Compose Multiplatform (Kotlin Multiplatform - KMP):** Reuses your Compose UI and Kotlin ViewModels on iOS. **2. Flutter (Dart):** Uses packages like `just_audio` and `audio_service` to drive native audio on both platforms.

9. Project Directory Tree Reference

```
c:\Users\edeep\suno\
├── gradle/
│   └── libs.versions.toml             # Modern Gradle Version Catalog
├── app/
│   ├── build.gradle.kts               # Module build script (Compose, Media3, Hilt, Room)
│   ├── src/main/
│   │   ├── AndroidManifest.xml        # Permissions & Foreground Service declarations
│   │   └── java/com/deepak/suno/
│   │       ├── data/
│   │       │   ├── local/             # Room Database, DAOs & Entities
│   │       │   ├── mediastore/        # Scoped Storage MediaStore scanner
│   │       │   └── repository/        # Clean Architecture Repository pattern
│   │       ├── di/                    # Hilt Dependency Injection modules
│   │       ├── domain/model/          # Song, Album, Artist, Playlist, PlayerState
│   │       ├── player/                # PlaybackService & MusicController IPC wrapper
│   │       ├── ui/
│   │       │   ├── components/        # SongListItem, MiniPlayer, PermissionPrompt
│   │       │   ├── screens/           # LibraryScreen, LibraryViewModel, NowPlayingSheet
│   │       │   └── theme/             # Color, Type, Material 3 Theme
│   │       └── MainActivity.kt        # Edge-to-edge Compose entry & permissions
└── Suno_Architecture_and_Implementation_Guide.pdf  # Generated Documentation
```

— Generated for Deepak (@deepak) • Suno Local Music Player Project —